

FINITE ELEMENTS IN PRACTICE · PART II

Introduction to FEniCS

Advanced PDE examples

Daniel Fernandez

Nonlinear · nonlocal · optimal control · coupled systems

What changes in Part II

We still write a weak form, discretize, and solve. Here the equations add nonlinear coefficients, nonlocal terms, controls, and coupled fields.

1. Nonlinear elliptic operator: 3D p -Laplacian
2. Nonlocal parabolic coefficient: diffusion depends on $\int_{\Omega} u$
3. Degenerate nonlocal null control: compute a source on ω
4. PDE-constrained optimization: state, adjoint, and control
5. Coupled nonlinear systems: reaction-diffusion pattern formation

Start with the residual

Mathematics

$$F(u; v) = 0 \quad \forall v \in V$$

$$J(u; \delta u, v) = \frac{d}{d\epsilon} F(u + \epsilon \delta u; v) \Big|_{\epsilon=0}$$

FEniCSx

```
1 u = fem.Function(V)
2 v = ufl.TestFunction(V)
3
4 F = ... # residual
5 J = ufl.derivative(F, u)
6
7 problem = NonlinearProblem(
8     F, u, bcs=[bc], J=J)
9 problem.solve()
```

EXAMPLE 7

3D p -Laplacian



A porous-flow model in three dimensions

Pressure-driven flow of a power-law fluid in a porous block

$$-\nabla \cdot \left(K(x)(\varepsilon^2 + |\nabla u|^2)^{\frac{p-2}{2}} \nabla u \right) + \sigma u = s(x) \quad \text{in } \Omega = (0,1)^3$$

$$K(x)(\varepsilon^2 + |\nabla u|^2)^{\frac{p-2}{2}} \nabla u \cdot n = 0 \quad \text{on } \partial\Omega.$$

- ▶ u : pressure head or hydraulic head in the 3D block.
- ▶ $-K(\varepsilon^2 + |\nabla u|^2)^{(p-2)/2} \nabla u$: nonlinear flux.
- ▶ $K(x)$: heterogeneous permeability of the medium.
- ▶ $s(x)$: injection/extraction wells or volumetric forcing.
- ▶ σu : small leakage/reference term for the Neumann model.
- ▶ $p = 2$: linear Darcy/Poisson model; $p \neq 2$: power-law rheology.

Weak form

Let $V = H^1(\Omega)$ and define

$$\mu(\nabla u) = (\varepsilon^2 + |\nabla u|^2)^{\frac{p-2}{2}}.$$

Find $u \in V$ such that

$$F(u; v) = \int_{\Omega} K(x) \mu(\nabla u) \nabla u \cdot \nabla v \, dx + \int_{\Omega} \sigma u v \, dx - \int_{\Omega} s v \, dx = 0 \quad \forall v \in V.$$

Regularization: $\varepsilon > 0$ regularizes the operator near $|\nabla u| = 0$. The demo uses $p = 3$ and homogeneous Neumann data. The σu term fixes the additive constant that a pure Neumann operator would otherwise leave free.

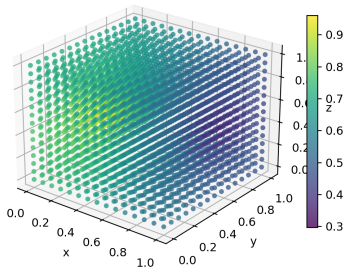
p-Laplacian: key code

```
1 msh = mesh.create_unit_cube(MPI.COMM_WORLD, 14, 14, 14)
2 V = fem.functionspace(msh, ("Lagrange", 1))
3
4 u = fem.Function(V, name="u")
5 v = ufl.TestFunction(V)
6 x = ufl.SpatialCoordinate(msh)
7
8 K = 0.7 + 0.6*x[2] + 0.15*ufl.sin(2.0*ufl.pi*x[0])
9 source = injection_well(x) - extraction_well(x)
10 sigma = 0.15
11
12 grad_u = ufl.grad(u)
13 mobility = (eps**2 + ufl.inner(grad_u, grad_u))**((p_exp-2.0)/2.0)
14
15 F = K*mobility*ufl.inner(grad_u, ufl.grad(v))*ufl.dx \
16     + sigma*u*v*ufl.dx \
17     - source*v*ufl.dx
18 J = ufl.derivative(F, u)
19
20 problem = NonlinearProblem(F, u, bcs=[], J=J,
21     petsc_options={"snes_type": "newtonls",
22     "ksp_type": "preonly", "pc_type": "lu"})
23 problem.solve()
```

snes_type=newtonls: Newton with line search. ksp_type=preonly, pc_type=lu: solve each Newton linear system directly by LU.

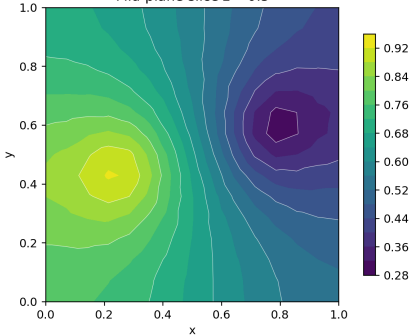
p-Laplacian: 3D result

3D pressure head

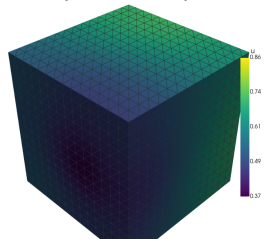


Regularized p-Laplacian in a 3D porous block

Mid-plane slice $z \approx 0.5$



Boundary mesh colored by solution



EXAMPLE 8

Nonlocal parabolic PDE



Nonlocal state equation

Problem

$$u_t - \left(a(x) \int_{\Omega} u(\xi, t) d\xi u_x \right)_x = h(x) \mathbf{1}_O(x), \quad \Omega = (0, 1), \quad t \in (0, T).$$
$$u(0, t) = u(1, t) = 0, \quad u(x, 0) = y_0(x).$$

- ▶ $M(t) = \int_0^1 u(x, t) dx$ couples every point to the global state.
- ▶ $a(x)$ is the local transport/diffusion profile.
- ▶ $h(x) \mathbf{1}_O$ is an actuator or forcing active only on O .
- ▶ Demo choice: $O = (0.35, 0.65)$ and several pairs (a, h) .

Backward Euler + Picard linearization

Let $M^{n+1} = \int_0^1 u^{n+1}(x) dx$. For $v \in H_0^1(0, 1)$,

$$(u^{n+1}, v) + \Delta t M^{n+1} (a u_x^{n+1}, v_x) = (u^n, v) + \Delta t (h \mathbf{1}_O, v).$$

Picard loop at each time step:

$$M_k = \int_0^1 u_k^{n+1} dx, \quad (u_{k+1}^{n+1}, v) + \Delta t M_k (a(u_{k+1}^{n+1})_x, v_x) = (u^n, v) + \Delta t (h \mathbf{1}_O, v).$$

This turns the nonlocal nonlinear step into repeated linear variational solves.

Nonlocal PDE: key code

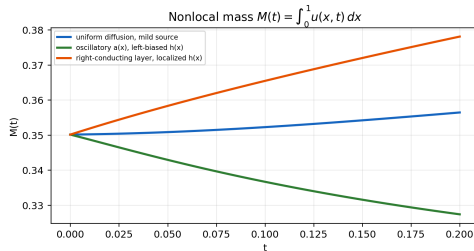
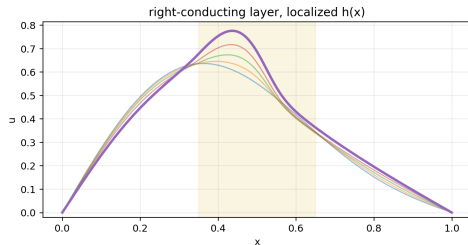
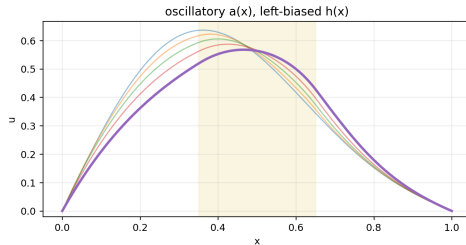
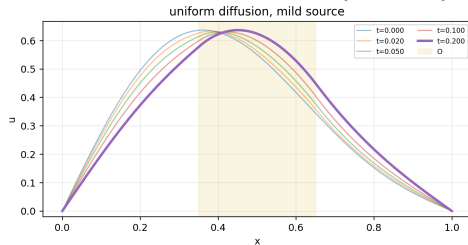
```
1 u = ufl.TrialFunction(V)
2 v = ufl.TestFunction(V)
3 M = fem.Constant(msh, default_scalar_type(0.0))
4 dt = fem.Constant(msh, default_scalar_type(DT))
5 a_form = u*v*ufl.dx \
6     + dt*M*a_fun*ufl.dot(ufl.grad(u), ufl.grad(v))*ufl.dx
7 L_form = u_n*v*ufl.dx + dt*h_fun*v*ufl.dx
8 problem = LinearProblem(a_form, L_form, bcs=[bc],
9     petsc_options={"ksp_type": "preonly", "pc_type": "lu"})
10
11 for step in range(1, num_steps + 1):
12     mass_k = assemble_integral(u_n)
13     u_picard = fem.Function(V)
14     u_picard.x.array[:] = u_n.x.array
15     for k in range(max_picard):
16         M.value = default_scalar_type(mass_k)
17         u_new = problem.solve() # one linear solve per Picard step
18         mass_new = assemble_integral(u_new)
19         update = max(abs(u_new.x.array - u_picard.x.array))
20         if update < update_tol and abs(mass_new - mass_k) < mass_tol:
21             break
22         u_picard.x.array[:] = u_new.x.array; mass_k = mass_new
23     u_n.x.array[:] = u_new.x.array
```

Nonlocal PDE: tested choices of a and h

Case	$a(x)$	$h(x)$ on O
Uniform	0.28	1.15
Oscillatory	$0.22(1 + 0.55 \sin(2\pi x)) + 0.04$	$1.25(1 + 0.60 \cos(2\pi x))$
Layered	$0.08 + \frac{0.34}{1 + \exp(-45(x - 0.58))}$	$2 \exp(-95(x - 0.48)^2)$

Nonlocal PDE: varying a and h

Nonlocal parabolic equation: effect of $a(x)$ and $h(x)$



EXAMPLE 9

Degenerate nonlocal null control



Degenerate nonlocal state equation

Controlled equation

$$u_t - \ell\left(\int_0^1 u(x, t) dx\right) (x^\gamma u_x)_x - 20t u = h(t, x) \mathbf{1}_\omega(x), \quad (t, x) \in (0, T) \times (0, 1),$$
$$u(0, x) = 2 \sin(2\pi x), \quad \ell(r) = 1 + \frac{1}{1 + r^2}.$$

- ▶ $\gamma \in \{0.5, 1, 1.5\}$ controls the strength of degeneracy at $x = 0$.
- ▶ Weak case $\gamma < 1$: $u(t, 0) = u(t, 1) = 0$.
- ▶ Strong case $\gamma \geq 1$: $(x^\gamma u_x)(t, 0) = 0$ and $u(t, 1) = 0$.
- ▶ The goal is to choose h on ω so that $u(T) \approx 0$.

Control domains and numerical loop

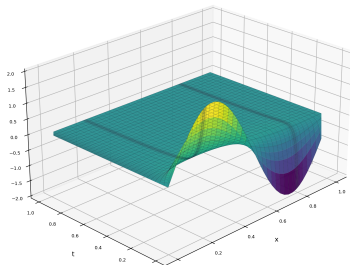
Domain	ω	ω'
1	(0.2, 0.8)	(0.21, 0.79)
2	(0.2, 0.8)	(0.3, 0.7)
3	(0.4, 0.6)	(0.42, 0.58)

1. Backward Euler in time and finite differences in space.
2. Newton solves the fully nonlinear time step, including the derivative of $\ell(\int u)$.
3. A frozen linear null control is computed from the discrete controllability map.
4. A discrete adjoint refinement reduces the nonlinear terminal error.

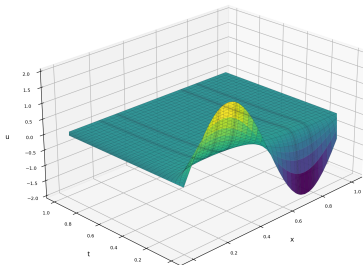
Run: $T = 1$, a feature-refined spatial grid with 60 target intervals, and 120 time steps. Nodes cluster near $x = 0$ and the interfaces of ω, ω' .

3D surface with mesh: $\gamma = 0.5$

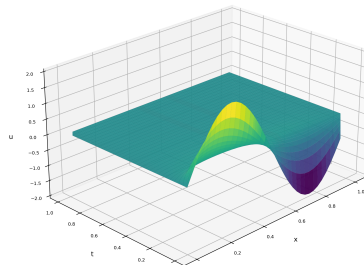
Domain 1: $\omega = (0.2, 0.8)$



Domain 2: $\omega = (0.2, 0.8)$

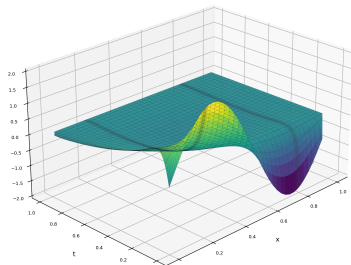


Domain 3: $\omega = (0.4, 0.6)$

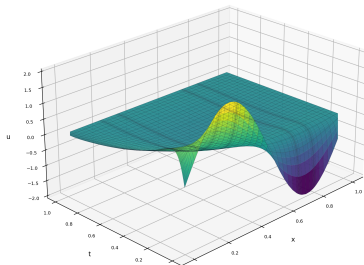


3D surface with mesh: $\gamma = 1$

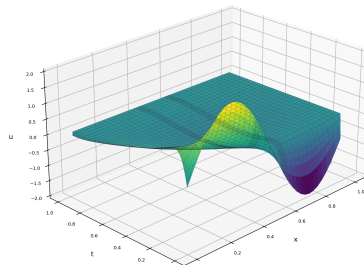
Domain 1: $\omega = (0.2, 0.8)$



Domain 2: $\omega = (0.2, 0.8)$

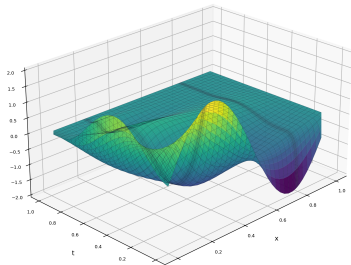


Domain 3: $\omega = (0.4, 0.6)$

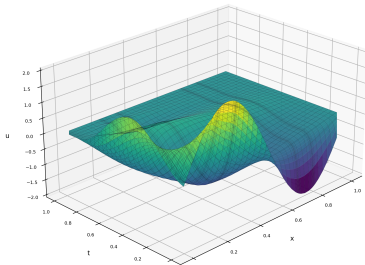


3D surface with mesh: $\gamma = 1.5$

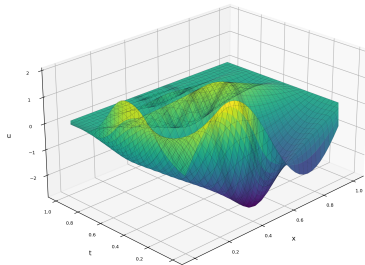
Domain 1: $\omega = (0.2, 0.8)$



Domain 2: $\omega = (0.2, 0.8)$

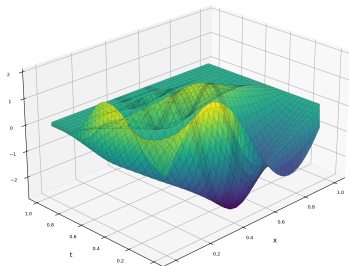


Domain 3: $\omega = (0.4, 0.6)$

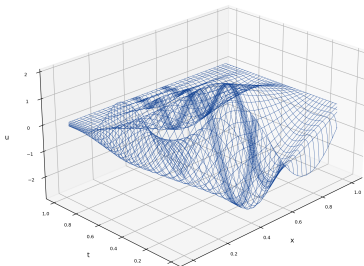


3D visualization variants

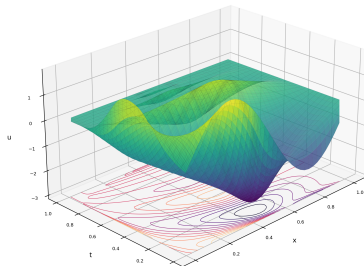
Surface with mesh overlay



Wireframe mesh



Surface with contour projection



Three views of the same solution: a surface with mesh edges, a wireframe, and a surface with contours.

EXAMPLE 10

Optimal control of a heat equation with drift



Heat equation with drift as a state constraint

Distributed source-control problem

$$\min_f J(f) = \frac{1}{2} \|y(T) - y_T\|_{L^2(\Omega)}^2 + \frac{\alpha}{2} \int_0^T \|f(t)\|_{L^2(\Omega)}^2 dt$$

subject to

$$y_t - \nu \Delta y + \beta \cdot \nabla y = f, \quad y = 0 \text{ on } \partial\Omega, \quad y(0) = y_0.$$

- ▶ y : temperature or concentration.
- ▶ $\nu \Delta y$: diffusion/smoothing.
- ▶ $\beta \cdot \nabla y$: drift/advection by a background flow.
- ▶ f : distributed heater, source, or actuator to be optimized.

State, adjoint, optimality

For final-time tracking, the first-order system is

$$\begin{aligned}y_t - \nu \Delta y + \beta \cdot \nabla y &= f, & y(0) &= y_0, \\ -p_t - \nu \Delta p - \beta \cdot \nabla p &= 0, & p(T) &= y(T) - y_T, \\ \alpha f + p &= 0.\end{aligned}$$

Workflow:

1. Solve the state equation forward in time.
2. Solve the adjoint equation backward in time.
3. Update the source control using the reduced gradient $g = \alpha f + p$.

With a running tracking term $\frac{1}{2} \int_0^T \|y - y_d\|^2 dt$, the adjoint gets a source term $y - y_d$.

Why $g = \alpha f + p$?

For the state produced by the current control f , the objective is

$$J(f) = \frac{1}{2} \|y(T) - y_T\|_{L^2(\Omega)}^2 + \frac{\alpha}{2} \int_0^T \|f(t)\|_{L^2(\Omega)}^2 dt.$$

For a perturbation δf , the state variation δy solves the linearized state equation with right-hand side δf . Testing that linearized equation with the adjoint p and integrating by parts gives

$$(y(T) - y_T, \delta y(T)) = \int_0^T (p, \delta f)_{L^2(\Omega)} dt.$$

Therefore

$$J'(f)\delta f = \int_0^T (\alpha f + p, \delta f)_{L^2(\Omega)} dt, \quad g = \nabla J(f) = \alpha f + p.$$

One backward adjoint solve gives the whole gradient, regardless of the number of control variables. Alternatives include finite differences, automatic differentiation, and a direct solve of the full KKT system.

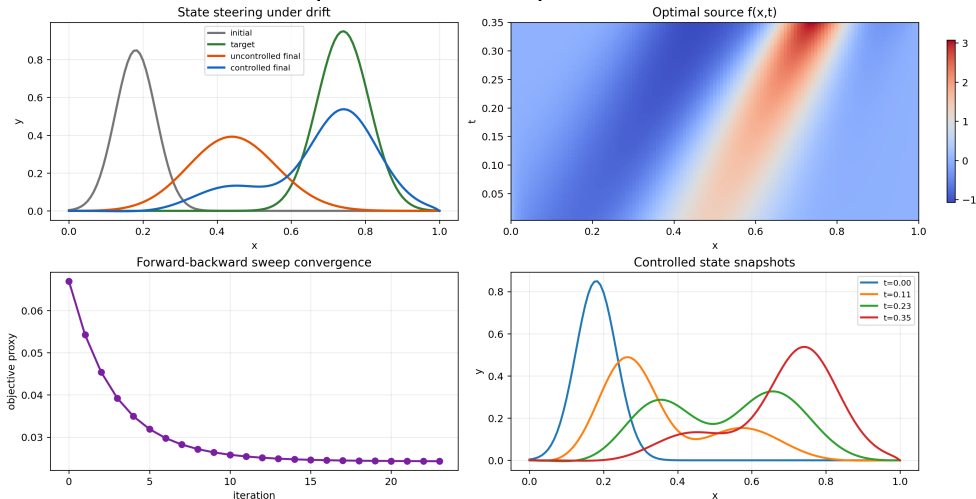
Optimal control loop: key code

```
1 controls = [fem.Function(V) for n in range(num_steps)]
2
3 for it in range(control_iters):
4     # 1. forward state solve
5     states = solve_state(y0, controls)
6
7     # 2. backward adjoint solve
8     p_T.x.array[:] = states[-1].x.array - target.x.array
9     adjoints = solve_adjoint(p_T)
10
11     # 3. reduced-gradient update
12     for n, f_n in enumerate(controls):
13         gradient = alpha*f_n.x.array + adjoints[n+1].x.array
14         f_n.x.array[:] = f_n.x.array - step_size*gradient
```

Reuse the assembled heat operators. Only the right-hand sides change during the forward and backward solves.

Optimal control: result

Optimal control of a heat equation with drift



EXAMPLE 11

Coupled reaction-diffusion system



Coupled nonlinear reaction-diffusion system

Gray-Scott activator-substrate model

$$\begin{aligned}u_t - D_u \Delta u &= -uv^2 + F(1 - u), \\v_t - D_v \Delta v &= uv^2 - (F + k)v, \end{aligned} \quad \partial_n u = \partial_n v = 0.$$

- ▶ u : feed/substrate concentration.
- ▶ v : autocatalytic species concentration.
- ▶ uv^2 : nonlinear reaction coupling the two equations.
- ▶ $F(1 - u)$: feed replenishment; $(F + k)v$: removal/decay.
- ▶ Unequal diffusion and nonlinear reaction can create spatial patterns.

Mixed weak form

Use $W = V \times V$ and test functions (ϕ, ψ) . At a backward Euler step, find $(u^{n+1}, v^{n+1}) \in W$ such that

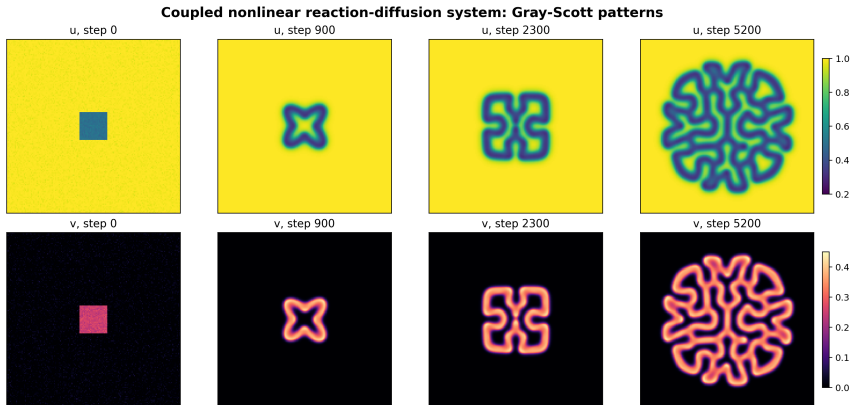
$$\begin{aligned} 0 = & \left(\frac{u^{n+1} - u^n}{\Delta t}, \phi \right) + D_u(\nabla u^{n+1}, \nabla \phi) - (-u^{n+1}(v^{n+1})^2 + F(1 - u^{n+1}), \phi) \\ & + \left(\frac{v^{n+1} - v^n}{\Delta t}, \psi \right) + D_v(\nabla v^{n+1}, \nabla \psi) - (u^{n+1}(v^{n+1})^2 - (F + k)v^{n+1}, \psi). \end{aligned}$$

The homogeneous Neumann condition is natural: no boundary constraint is needed.

Reaction-diffusion: FEniCS residual

```
1 P1 = basix.ufl.element("Lagrange", msh.basix_cell(), 1)
2 W = fem.functionspace(msh, basix.ufl.mixed_element([P1, P1]))
3
4 w = fem.Function(W)          # current unknown pair
5 w_old = fem.Function(W)      # previous time level
6 u, v = ufl.split(w)
7 u0, v0 = ufl.split(w_old)
8 phi, psi = ufl.TestFunctions(W)
9
10 R_u = -u*v*v + F_feed*(1.0-u)
11 R_v = u*v*v - (F_feed+k_kill)*v
12
13 F = ((u-u0)/dt)*phi*ufl.dx + D_u*ufl.inner(ufl.grad(u), ufl.grad(phi))*ufl.dx \
14     - R_u*phi*ufl.dx \
15     + ((v-v0)/dt)*psi*ufl.dx + D_v*ufl.inner(ufl.grad(v), ufl.grad(psi))*ufl.dx \
16     - R_v*psi*ufl.dx
17 J = ufl.derivative(F, w)
18
19 problem = NonlinearProblem(F, w, bcs=[], J=J,
20     petsc_options={"snes_type": "newtonls",
21         "ksp_type": "preonly", "pc_type": "lu"})
22
23 for step in range(num_steps):
24     problem.solve()
25     w_old.x.array[:] = w.x.array
```

Reaction-diffusion: pattern formation



The nonlinearity transfers mass between the two species while diffusion spreads each species at a different rate. Together they produce the patterns shown here.

Solving the five problem types

- ▶ Nonlinear PDEs: write $F(u; v) = 0$ and let UFL differentiate the Jacobian.
- ▶ Nonlocal PDEs: assemble global quantities, then solve linearized steps.
- ▶ Degenerate control problems: combine linearized controllability maps with adjoint refinement.
- ▶ Optimal control: solve the state forward and the adjoint backward to compute the gradient.
- ▶ Coupled systems: use mixed spaces and solve for all fields together.

Questions?



Backup: rebuild figures

```
1 conda run -n fenicsx-env python experiments/10_plaplacian_3d.py
2 conda run -n fenicsx-env python experiments/11_nonlocal_variants.py
3 python experiments/14_degenerate_nonlocal_control.py --mesh feature --nx 60 --nt 120 --refine-iters 50
4 conda run -n fenicsx-env python experiments/12_heat_drift_control.py
5 conda run -n fenicsx-env python experiments/13_reaction_diffusion_system.py
```

Compile the deck:

```
1 latexmk -pdf -interaction=nonstopmode 1.2_fenics_part2.tex
```